Sprawozdanie – Laboratorium 1: robot Unitree Go2 EDU

Autor: Wojciech Maćkowiak **Data zajęć:** 05.10.2026

1. Cel ćwiczenia

Celem zajęć było zapoznanie się z robotem Unitree Go2 EDU: jego architekturą (w tym modułem NVIDIA Jetson), systemem i komunikacją (Ubuntu, Cyclone DDS, ROS 2 Foxy i Humble w Dockerze), monitorem do zarządzania aplikacjami oraz sensorami – wbudowanymi (kamera RGB, LiDAR) i dodatkowymi (Intel RealSense D435, Hesai XT16). W drugiej części omawiam scenariusz z człowiekiem idącym w stronę robota oraz wybór między MediaPipe a YOLO do detekcji szkieletu.

2. Architektura robota

Go2 to robot czworonożny o masie ok. 15 kg i 12 stopniach swobody (3 silniki na nogę). Wersja EDU różni się od Air i Pro przede wszystkim tym, że daje dostęp do sterowania niskiego poziomu (bezpośrednio silnikami), ma czujniki nacisku w stopach i moduł rozszerzeń z komputerem NVIDIA Jetson Orin.

W robocie są w praktyce dwa komputery połączone wewnętrzną siecią Ethernet (podsieć 192.168.123.0/24):

- **wewnętrzny sterownik ruchu** (adres .161) – odpowiada za pętlę sterowania silnikami, balans, estymację stanu z IMU i enkoderów oraz tryb „sport", czyli chodzenie z zadaną prędkością. Obsługuje też wbudowaną kamerę i LiDAR;
- **Jetson Orin** (adres .18) – komputer dla użytkownika, na którym uruchamia się własne programy: percepcję, sieci neuronowe, sterowniki dodatkowych sensorów. Ma GPU, więc można na nim uruchamiać modele wizyjne (np. przez TensorRT).

Taki podział jest dobry z punktu widzenia bezpieczeństwa. Nawet jeśli nasz program na Jetsonie się zawiesi albo mocno obciąży procesor, robot się nie przewróci, bo lokomocja działa na osobnym sterowniku.

Żeby połączyć się z robotem z laptopa, ustawiłem na karcie sieciowej statyczny adres z tej samej podsieci (np. 192.168.123.99/24). Po tym da się zrobić `ping` do obu komputerów i zalogować się na Jetsona przez SSH.

3. Oprogramowanie

3.1. Ubuntu i ROS 2 Foxy

Na Jetsonie działa Ubuntu 20.04 (L4T z JetPacka 5). Z tego wynika wersja ROS 2: na tej wersji Ubuntu dostępne jest Foxy i pod nie przygotowane są pakiety

Unitree (`unitree_ros2`). Problem w tym, że Foxy nie jest już wspierane (EOL w 2023 r.), a nowsze wersje wielu pakietów, np. sterowników RealSense i Hesai czy Nav2, są robione pod Humble.

3.2. Cyclone DDS

ROS 2 do komunikacji używa DDS. Unitree oparło całą komunikację robota na implementacji Cyclone DDS (wersja 0.10.x). Do robota można więc podłączyć się albo bezpośrednio przez `unitree_sdk2`, albo przez ROS 2, ale tylko wtedy, gdy ROS używa Cyclone. Domyślnym RMW w Foxy i Humble jest Fast DDS, więc trzeba to ustawić ręcznie. Trzeba też wskazać interfejs sieciowy, bo inaczej discovery (multicast) potrafi pójść przez zły interfejs, np. Wi-Fi albo docker0, i tematy robota się nie pokażą:

```
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export CYCLONEDDS_URI='<CycloneDDS><Domain><General><Interfaces>
  <NetworkInterface name="eth0" priority="default" multicast="default"/>
</Interfaces></General></Domain></CycloneDDS>'
```

W Foxy Cyclone w wersji 0.10 trzeba zbudować ze źródeł, bo pakiet binarny ma starszą wersję. W Humble wystarczy pakiet `ros-humble-rmw-cyclonedds-cpp`. Po poprawnej konfiguracji `ros2 topic list` pokazuje m.in. `/sportmodestate` (pozycja i prędkość robota), `/lowstate` (stan silników, IMU, bateria), `/api/sport/request` (polecenia ruchu) i `/utlidar/cloud` (chmura z LiDARa).

3.3. Docker z Ubuntu 22.04 i ROS 2 Humble

Żeby używać Humble bez zmiany systemu na Jetsonie, uruchamia się je w kontenerze z Ubuntu 22.04. Kontener musi mieć `--network host`, bo przez domyślny most Dockera multicast DDS nie przechodzi. Potrzebny jest też dostęp do urządzeń (`--privileged` lub `-v /dev:/dev` dla kamery USB) i do GPU (`--runtime nvidia`). W kontenerze trzeba ustawić te same zmienne Cyclone co na hoście i zbudować wiadomości Unitree (`unitree_go`, `unitree_api`). Oficjalnie Foxy i Humble nie muszą się ze sobą komunikować. W praktyce przy tym samym Cyclone i tych samych definicjach wiadomości zwykłe tematy działają, a problemy pojawiają się raczej przy serwisach i akcjach.

3.4. Monitor z zarządcą zdarzeń i aplikacji

[Do uzupełnienia – opis narzędzia pokazanego na zajęciach: jak się nazywa, jak wygląda, co pozwala uruchamiać i jakie zdarzenia obsługuje.]

Takie narzędzie jest przydatne, bo system składa się z wielu procesów na dwóch komputerach i w kontenerach. Z jednego miejsca widać stan robota (bateria, temperatury, czy dane z sensorów przychodzą), można uruchamiać i zatrzymywać poszczególne aplikacje, np. sterowniki sensorów i detekcję, oraz reagować na zdarzenia, np. zatrzymać robota, gdy przestaną przychodzić dane z kamery.

4. Sensory

Kamera RGB Go2. Szerokokątna kamera z przodu robota. Obraz jest przesyłany jako strumień H.264 przez UDP multicast i można go odebrać GStreamerem lub przez `unitree_sdk2`. Nie daje głębi, ma dużą dystorsję (trzeba ją skalibrować) i pewne opóźnienie przez kompresję. Do tego obraz drga i się rozmywa podczas chodu.

LiDAR Unitree L1. Wbudowany LiDAR o polu widzenia zbliżonym do półsfery (ok. 360° × 90°), widzący już od kilku centymetrów. Chmura jest jednak rzadka (ok. 21 tys. punktów/s). Dobrze nadaje się do wykrywania przeszkód tuż przy robocie, gorzej do śledzenia ludzi na większych odległościach.

Intel RealSense D435. Kamera głębi oparta na aktywnej stereowizji w podczerwieni, z dodatkową kamerą RGB. Głębia ma do 1280 × 720 pikseli i pole widzenia ok. 87° × 58°. Najlepiej działa w zakresie ok. 0,3–3 m, a błąd rośnie z kwadratem odległości. W ROS 2 ważna jest opcja wyrównania głębi do obrazu RGB (`align_depth`). Dzięki niej dla punktu wykrytego na obrazie, np. stawu szkieletu, można odczytać odległość i przeliczyć go na punkt 3D. Wady: krótki zasięg i problemy z powierzchniami bez tekstury lub błyszczącymi.

Hesai XT16. 16-kanałowy obrotowy LiDAR o polu widzenia 360° × 30° i zasięgu do ok. 100 m z dokładnością ok. 1 cm. Daje dużo gęstszą i dokładniejszą chmurę niż L1, więc nadaje się do SLAM-u i do śledzenia ludzi z większej odległości. Minusem jest wąskie pole w pionie. Przy montażu na wysokości ok. 0,5 m najniższa wiązka (−15°) dochodzi do podłogi dopiero ok. 1,9 m od robota, więc niskich przeszkód bliżej robota nie widać. Tu przydaje się L1.

Sensor	Zasięg	Głębia	Do czego najlepiej
Kamera Go2	duży	nie	detekcja człowieka/szkieletu
L1	mały–średni	rzadka	przeszkody blisko robota
D435	0,3–3 m	gęsta	położenie 3D człowieka z bliska
XT16	do ~100 m	dokładna	SLAM, śledzenie ludzi z daleka

Żeby łączyć dane z tych sensorów, trzeba znać ich wzajemne położenie (kalibracja i drzewo transformacji tf2) i synchronizować czas. Inaczej przy ruchu robota położenie człowieka będzie wyraźnie przesunięte.

5. Prace naukowe z wykorzystaniem Go2

Go2 i jego poprzednik Go1 są dziś bardzo popularne w badaniach, głównie dlatego, że są dużo tańsze od robotów typu ANYmal czy Spot, a wersja EDU daje dostęp do sterowania silnikami.

Najwięcej prac dotyczy **uczenia chodu metodami RL w symulacji i przenoszenia na prawdziwego robota**. Podstawą jest praca Rudina i in. (2022), w której politykę chodu trenuje się w kilkanaście minut dzięki tysiącom

równoległych symulacji na GPU. Unitree udostępnia na tej bazie repozytorium `unitree_rl_gym` z konfiguracją dla Go2, a Go2 jest też dostępny w Isaac Lab.

Kolejny kierunek to **manipulacja**. W pracy „UMI on Legs" (Ha i in., 2024) Go2 z zamontowanym ramieniem wykonuje zadania manipulacyjne wyuczone z demonstracji ludzi. Go2 pojawia się też w pracach o **nawigacji z modelami językowymi**, np. NaVILA (Cheng i in., 2024). Tam duży model wizyjno-językowy decyduje, gdzie iść, a osobna polityka RL odpowiada za sam chód.

Z punktu widzenia tego laboratorium najciekawsza była praca „Agile But Safe" (He i in., 2024), wykonana na Go1. Robot ma w niej dwie polityki, szybką i „ratunkową", a sieć neuronowa ocenia, czy przy aktualnej prędkości kolizji da się jeszcze uniknąć. Pokazuje to, że przy większych prędkościach trzeba przewidywać, co się stanie, a nie tylko reagować na to, co jest teraz.

6. Scenariusz: człowiek, robot i możliwa kolizja

[Do uzupełnienia – krótki opis tego, co dokładnie pokazano na zajęciach.]

W pokazanym scenariuszu robot i człowiek poruszają się jednocześnie, a ich drogi mogą się przeciąć. Najprostsze rozwiązanie jest reaktywne: robot zatrzymuje się, gdy człowiek jest bliżej niż np. 1 m. Moim zdaniem to nie wystarcza, z dwóch powodów.

Pierwszy powód to **opóźnienia**. Na łączny czas reakcji składają się: pobranie klatki i przesłanie strumienia (kilkadziesiąt ms), detekcja na Jetsonie (kilkadziesiąt ms), przesłanie polecenia i sama reakcja robota, który przy chodzie potrzebuje kilkuset ms, żeby zmienić prędkość. Według moich szacunków razem daje to ok. 0,3–0,7 s. Jeśli człowiek idzie z prędkością ok. 1,4 m/s, a robot jedzie ok. 1 m/s naprzeciw, to w tym czasie zbliżą się o 0,7–1,7 m. Próg 1 m przy takich prędkościach jest więc za mały.

Drugi powód to **brak informacji o kierunku ruchu**. Sama odległość nie mówi, czy człowiek się zbliża, czy oddala. Osoba oddalająca się na 0,9 m niepotrzebnie zatrzyma robota. Z kolei osoba idąca szybko w poprzek toru robota z 3 m nie wywoła żadnej reakcji, choć za 2 sekundy dojdzie do zderzenia.

Dlatego sensowniejsza jest **predykcja ruchu**. Najprostszy model zakłada, że przez najbliższe 1–2 s człowiek i robot poruszają się ze stałą prędkością. Dla położenia względnego $\mathbf{p} = \mathbf{p}_h - \mathbf{p}_r$ i prędkości względnej $\mathbf{v} = \mathbf{v}_h - \mathbf{v}_r$ najmniejsza odległość wypada w chwili

$$t^* = -\frac{\mathbf{p} \cdot \mathbf{v}}{|\mathbf{v}|^2}, \qquad d^* = |\mathbf{p} + \mathbf{v}\, t^*|.$$

Jeśli t^* jest dodatnie i mniejsze od przyjętego horyzontu, a d^* jest mniejsze od sumy „promieni" człowieka i robota z marginesem, robot powinien zareagować. Jeśli $t^* < 0$, obiekty już się oddalają i reakcja nie jest potrzebna. Dla przykładu z człowiekiem idącym w poprzek (3 m przed robotem, 2 m w bok, 1,2 m/s)

wychodzi $t^* \approx 2{,}2$ s i $d^* \approx 0{,}3$ m. Kolizja jest więc przewidywana dużo wcześniej, niż zadziałałby prosty próg odległości.

Prędkość człowieka najlepiej szacować filtrem Kalmana na podstawie kilku kolejnych pomiarów. Trzeba przy tym pamiętać, że **pozycję człowieka należy przeliczyć do układu świata** (odometria robota z `/sportmodestate`), a nie liczyć w układzie kamery. Kamera jedzie razem z robotem i się kołysze, więc w jej układzie nawet stojący człowiek „porusza się". Do śledzenia tej samej osoby między klatkami potrzebny jest tracker (np. ByteTrack). Niepewność predykcji rośnie z horyzontem, więc dla dalszych chwil warto zwiększać margines bezpieczeństwa. Bardziej zaawansowane modele, np. social force albo sieci typu Social LSTM czy Trajectron++, potrafią przewidywać skręty i interakcje, ale do tego zadania model stałej prędkości jest dobrym punktem startowym.

Robot po wykryciu zagrożenia może zwolnić, zatrzymać się albo ominąć człowieka. Go2 może też chodzić bokiem, co ułatwia omijanie. Najbardziej naturalne wydaje mi się zwolnienie i przepuszczenie człowieka, bo jest przewidywalne dla osoby. Niezależnie od predykcji robot powinien mieć prostą warstwę awaryjną, np. stop na podstawie L1, gdy coś znajdzie się bardzo blisko, na wypadek błędu detektora. Najtrudniejsza jest sytuacja, gdy ktoś wychodzi zza rogu tuż przy robocie. Wtedy nie ma historii ruchu, a kamera może go jeszcze nie widzieć.

7. MediaPipe czy YOLO do detekcji szkieletu

Szkielet daje więcej niż sam prostokąt wokół osoby. Z ustawienia barków i bioder widać, w którą stronę człowiek jest zwrócony i dokąd prawdopodobnie pójdzie. Z ruchu nóg widać, czy idzie, czy stoi. Położenie kostek pozwala dokładniej określić, gdzie człowiek stoi na podłodze. Dlatego uważam, że detekcja szkieletu ma tu sens i pomaga w predykcji ruchu.

MediaPipe Pose (model BlazePose) zwraca 33 punkty, w tym dłonie i stopy, oraz przybliżone współrzędne 3D. Jest bardzo lekki i działa w czasie rzeczywistym nawet na CPU. Jest jednak zrobiony głównie z myślą o jednej osobie blisko kamery i słabo radzi sobie z kilkoma osobami, z małymi sylwetkami i z sytuacją, gdy widać tylko część ciała.

YOLO-Pose (np. YOLOv8/YOLO11-pose z Ultralytics) w jednym przebiegu sieci wykrywa wszystkie osoby i dla każdej zwraca 17 punktów w formacie COCO, każdy z własną pewnością. Da się go wyeksportować do TensorRT i uruchomić szybko na GPU Jetsona, a biblioteka ma wbudowany tracking. Minusy: mniej punktów (brak dłoni i stóp), tylko 2D (głębię trzeba brać z D435 albo LiDAR-a) oraz licencja AGPL.

	MediaPipe Pose	YOLO-Pose
Liczba osób	głównie jedna	wiele
Punkty	33, przybliżone 3D	17, 2D

	MediaPipe Pose	YOLO-Pose
Częściowa sylwetka	słabo	dobrze
Na Jetsonie	CPU	GPU (TensorRT)
Tracking	brak	wbudowany

Dla robota ważna jest jeszcze **niska pozycja kamery**. Kamera jest ok. 0,35 m nad podłogą, a kamera RGB D435 ma w pionie ok. $\pm 21°$. Górna krawędź obrazu w odległości d jest więc na wysokości ok. $0{,}35 + 0{,}38\,d$. Żeby zmieściła się cała osoba o wzroście 1,75 m, musi stać ok. 3,7 m od robota. W odległościach istotnych dla kolizji, poniżej 2 m, widać głównie nogi. YOLO lepiej sobie z tym radzi, bo wykrywa osobę także po części ciała. MediaPipe w takich sytuacjach często w ogóle nie znajduje osoby, bo jego detektor opiera się na głowie i tułowiu. Dodatkowo oba modele są uczone na zdjęciach robionych z wysokości oczu, a nie z dołu.

Podsumowując: do scenariusza nawigacyjnego lepiej nadaje się YOLO-Pose, z głębią z D435, trackingiem i filtrem Kalmana. MediaPipe ma sens do interakcji z jedną osobą z bliska, np. do rozpoznawania gestów („stop”, wskazanie kierunku), albo jako dodatek uruchamiany na wycinku obrazu z osobą wykrytą przez YOLO.

8. Wnioski

- Go2 EDU oddziela sterowanie chodem (wewnętrzny sterownik) od obliczeń użytkownika (Jetson), co jest bezpieczne i wygodne przy pisaniu własnych aplikacji.
- Cała komunikacja opiera się na Cyclone DDS. Najczęstszy problem z „niewidocznymi” tematami wynika ze złego RMW albo złego interfejsu sieciowego.
- Docker z Humble pozwala używać nowszych pakietów na Jetsonie z Foxy, ale wymaga `--network host` i zbudowania wiadomości Unitree w kontenerze.
- Sensory się uzupełniają: kamera rozpoznaje, co jest przed robotem, D435 i XT16 dają odległości, a L1 pilnuje najbliższego otoczenia.
- W scenariuszu z człowiekiem sama reakcja na odległość nie wystarcza ze względu na opóźnienia rzędu 0,3–0,7 s. Potrzebna jest predykcja ruchu (choćby model stałej prędkości z filtrem Kalmana), a jako zabezpieczenie prosta warstwa awaryjna.
- Do detekcji szkieletu na robocie lepiej sprawdza się YOLO-Pose, a MediaPipe nadaje się raczej do gestów z bliska.

Literatura

1. Unitree Robotics – dokumentacja Go2 oraz repozytoria `unitree_sdk2`, `unitree_ros2`, `unitree_rl_gym` (github.com/unitreerobotics).

2. Dokumentacja Cyclone DDS (cyclonedds.io) i ROS 2 (docs.ros.org).
3. Karty katalogowe Intel RealSense D435 i Hesai PandarXT-16.
4. N. Rudin i in., „Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning", CoRL 2022.
5. H. Ha i in., „UMI on Legs: Making Manipulation Policies Mobile with Manipulation-Centric Whole-body Controllers", CoRL 2024.
6. A.-C. Cheng i in., „NaVILA: Legged Robot Vision-Language-Action Model for Navigation", arXiv:2412.04453, 2024.
7. T. He i in., „Agile But Safe: Learning Collision-Free High-Speed Legged Locomotion", RSS 2024.
8. V. Bazarevsky i in., „BlazePose: On-device Real-time Body Pose Tracking", arXiv:2006.10204, 2020.
9. D. Maji i in., „YOLO-Pose: Enhancing YOLO for Multi Person Pose Estimation Using Object Keypoint Similarity Loss", CVPR Workshops 2022.
10. Y. Zhang i in., „ByteTrack: Multi-Object Tracking by Associating Every Detection Box", ECCV 2022.